

XONIX GAME

Group 35

Masab Tahir 23i-0006

Muhammad Arslan 23i-0572

Section: C

Data Structures

Submitted to: Mr. Zahoor Hassan

Dated: May 11, 2025

1. Introduction

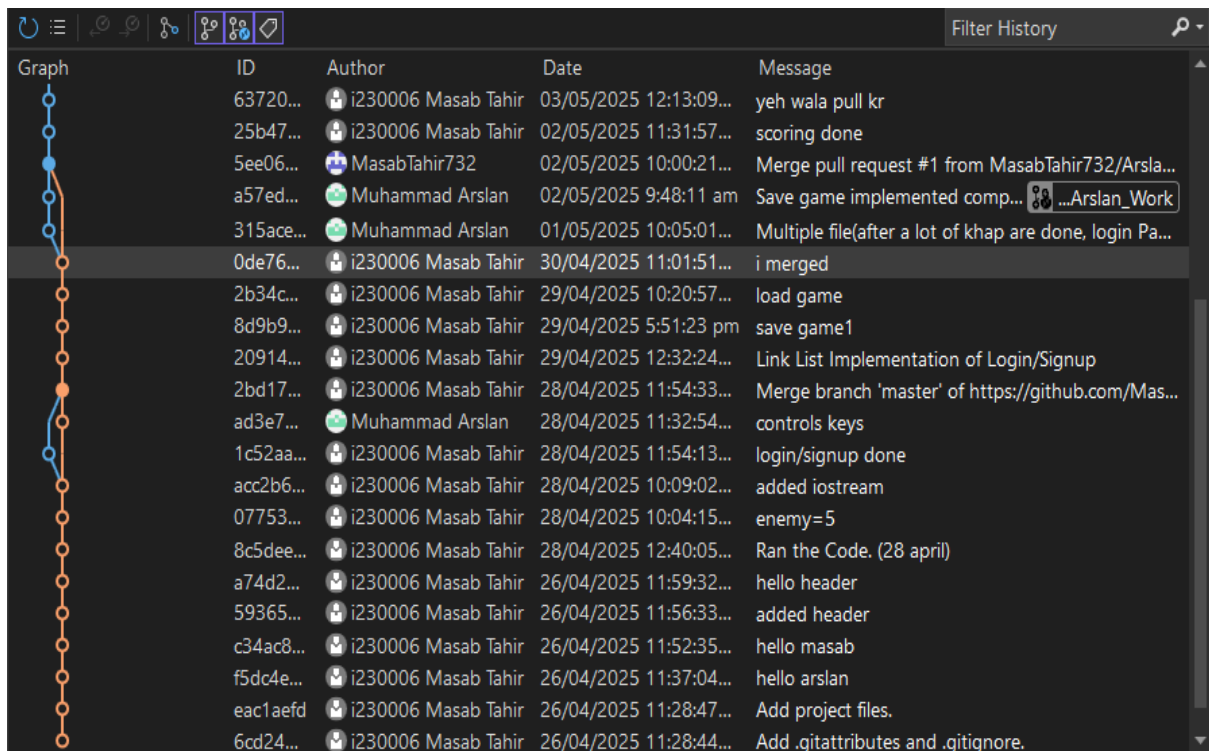
Overview and Features:

This project efficiently implements the Xonix Game along with the management of the game using Data Structures principles that we've studied and implemented in our labs. This game showcases our grip on Data Structures principles. This game has a player that has to capture the maximum area on a 2D grid, while keeping himself safe from the enemies. The game also has an option for the multiplayer in which both the players have to capture the area and the player with maximum captured area wins. The game stores the player's profile and also has a Match-Making Mechanism along with the Friend System and Themes. A player can also save his current progress using a save ID.



Tools/Languages Used

- **C++** – Core language used for game logic and systems programming
- **Git & GitHub** – A shared GitHub repository was created to share code and track each other's progress efficiently. It allowed us to push and pull code updates, track each other's progress, manage version control, and resolve merge conflicts when needed.
- **SFML (Simple and Fast Multimedia Library)** – Used for graphics rendering, input handling, and window management.
- **Notepad++ / VS Code** – For editing of the code and development.
- **Manual Testing & Debugging Tools** – Manually predicting the bug and then track it by using debugger and cout statements.
- **Online Documentation & Forums** – Visited SFML website to understand the working mechanism of the SFML functions and features.



2. Work Distribution

Below is the table that visualises the work distribution.

Task	Masab	Arslan
Login/Authentication	✓	
Main Menu/ Inventory System/		✓
Player Profile/Save Game Progress/ Scoring	✓	
Multi Player/ Report	✓	
Friend System/ Match Making Queue		✓

3. Workflow & Implementation Timeline

First of all, we analysed that which approach will be the best for the development of the project. After that, we setup the SFML and GitHub repository so that our work goes smooth. We dedicated 1-2 days for the understanding of the given code so that we can know what the current code is doing because we cannot initiate without understanding the current implementation. After the understanding of the code, we divided the task so that our work goes in a sequence. Masab Tahir did all the work that was related to the current code like creating Player Profile, Login/Authentication, Saving game grid and Scoring, whereas Arslan continued work on the addition new features like inventory and Main Menu. We used to Push our work in the shared repository daily so that we can track each other's work and merge easily. We also used to ensure that our group member has the same burden that we have. This ensured the group collaboration and teamwork.

4. Data Structures Used

Since this project was part of our Data Structures course, our primary focus was on deeply understanding and correctly implementing the required data structures. Instead of aiming for high marks or prioritizing visual polish or game complexity, we focused on the core learning objectives of the course—namely, designing and applying data structures effectively within a working application.

a. **Linked List**

Used for managing players and saving the game grid efficiently. This helped in dynamic memory management and flexible data insertion/deletion.

b. **Hash Table**

Implemented to enable fast searching of players during friend request handling, ensuring constant-time lookup for usernames.

c. **AVL Tree**

Used to store and manage themes in a balanced way, allowing for efficient insertion, deletion, and lookup operations while keeping the structure sorted.

d. **Priority Queue**

Applied in the matchmaking system to ensure that players of similar skill levels are paired together, improving the fairness and competitiveness of the game.

5. Challenges Faced & Solutions

a. **Implementing Multiplayer Logic in a Grid-Based System**

Initially, integrating two players on the same grid led to overlapping trail and tile values, making it difficult to distinguish which part belonged to which player.

Solution: We used distinct grid values for each player's trail and tiles (e.g., 1 & 2

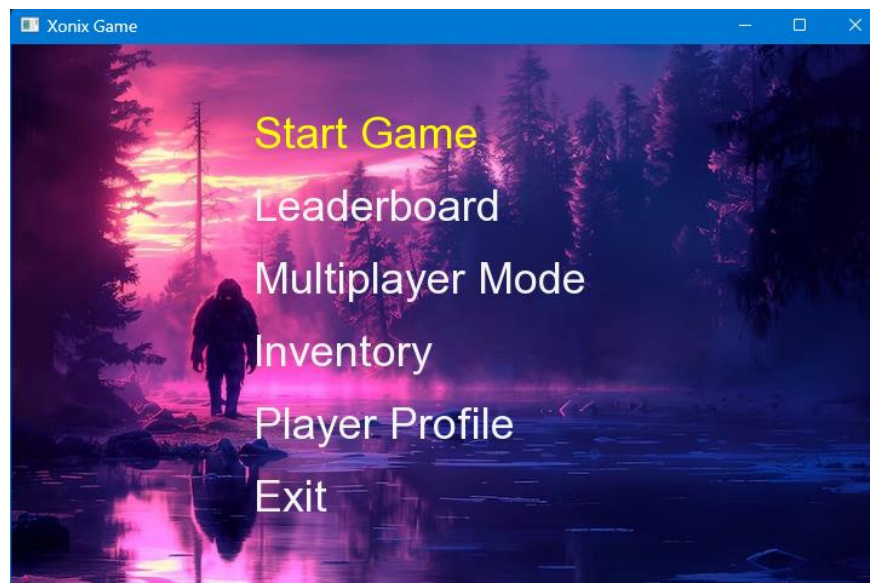
for Player 1, 3 & 4 for Player 2), which solved the confusion and allowed smooth updates and rendering.

b. Main Menu Implementation and shifting through pages

In our Main Menu implementation, we wrote a code and it was not going as we were expecting. Then we changed the implementation of the Main Menu and then implemented the other logic. The challenge was that even a single change in function arguments and window rendering caused a lot of bugs and syntax errors.

6. Screenshots or Sample Outputs

Main Menu:



Leader Board:



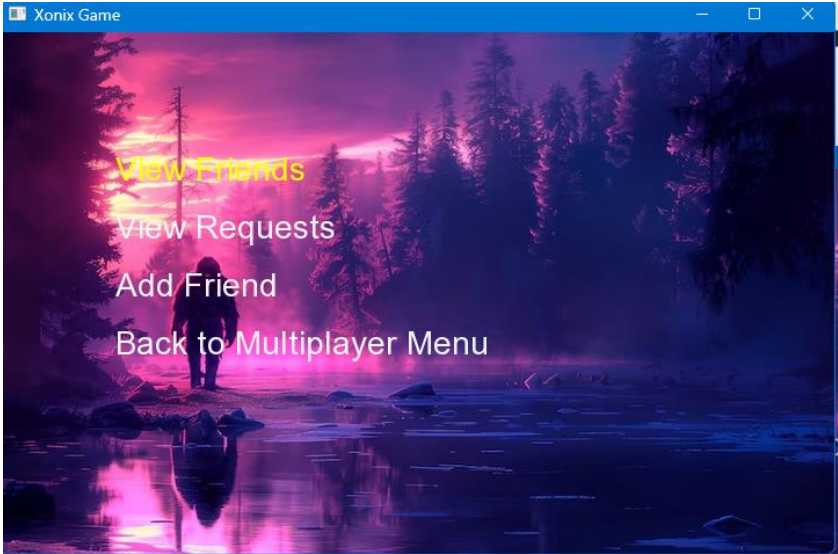
Theme Selection:



Player Profile:



Friend System:



Match Making Queue



Game Play:



7. Conclusion

In this project, we learned how to implement the theoretical concepts of Data Structures that we have studied in the class. Instead of focusing on visual presentation or maximizing marks, we gave priority to the correct implementation of data structures throughout the development process.

We used an **AVL tree** to manage and store game themes efficiently, that ensured quick and balanced access during gameplay without any issue. A **hash table** was implemented for rapid searching of the players, this allowed us to look for the user and efficiently accept and send friend request. For matchmaking, we used a **priority queue** to pair players of similar skill levels based on scores, this enhanced the fairness in the gameplay experience.

Besides technical learning, we also got a very fruitful experience in version control using **GitHub**, collaborative development, design, and debugging. This project made us realize the importance of planning in the group work, task division, and consistency in teamwork.

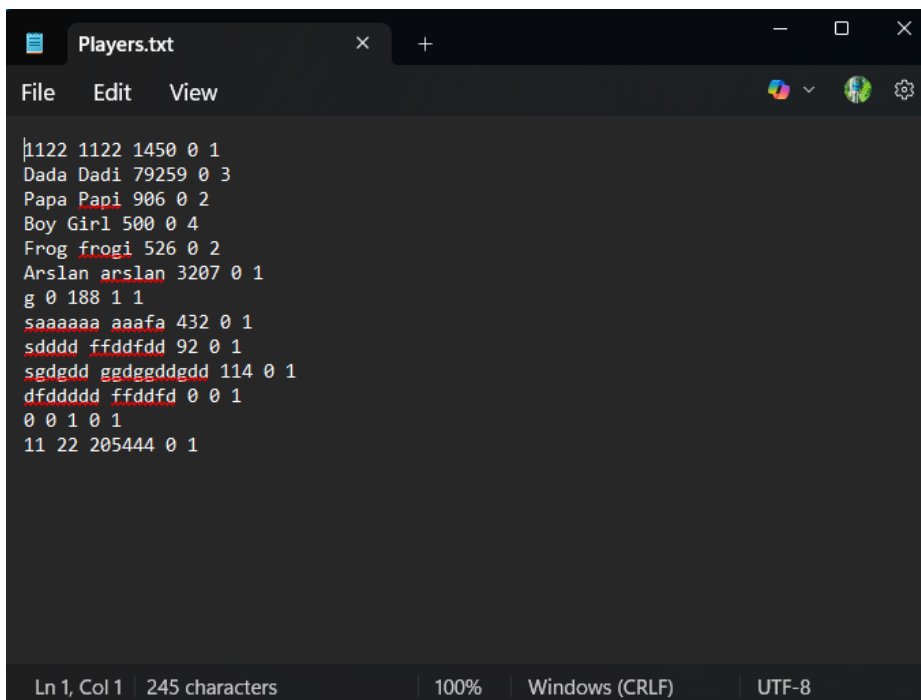
Suggestions for Improvements and Future Features:

- Introduce **online multiplayer** with real-time database implementation.
- Add more dynamic elements like power-ups and AI-based enemy behaviour.
- Expand the theme selection with user-unlockable content managed through the **AVL structure**.
- Improve UI with **sound, animations**, and better user **feedback systems**.

8. Appendix

A. Player Profile File Format

- 1st entry represents the Player's Username
- 2nd entry represents the Password
- 3rd entry represents the total score of the Player
- 4th entry represents the high score
- 5th entry represents the preferred theme of the player



```
1122 1122 1450 0 1
Dada Dadi 79259 0 3
Papa Papi 906 0 2
Boy Girl 500 0 4
Frog frogi 526 0 2
Arslan arslan 3207 0 1
g 0 188 1 1
saaaaaa aaafa 432 0 1
sdddd ffddfd 92 0 1
sgdggd ggdggdggd 114 0 1
dfddddd ffddfd 0 0 1
0 0 1 0 1
11 22 205444 0 1
```


B. Grid Saving File Format

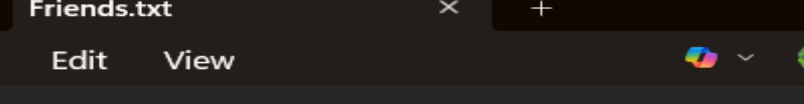
- The first line represents the user that is saving the game.
- The second line represents the Save ID entered by the user.
- Third line represents the timestamp at which the game was saved
- And then there's a grid that in which 1's represents the captured tiles and the 0's represents the empty spaces.

[illegible]

C. Friends File Format

Each line stores the data of a player for the friends.

- The first entry refers to the player whose friends are to be displayed in that line.
- The second and the entries onwards are the friends of the player



```
Dada: Arslan 11
Papa:
Boy: 11 Arslan
Frog:
Arslan: Dada 11 Boy
11: Dada Boy Arslan
```

D. Themes Struct

```
struct Themes {  
    int id;  
    sf::Color backgroundColor;  
    sf::Color textColor;  
    string font;  
    Themes* llink;  
    Themes* rlink;  
    int bfactor;  
    string image;  
};
```

E. Player Profile Node

```
class Node {  
public:  
    string Username;  
    string Password;  
    int TotalScore;  
    float WinPercentage;  
    Node* next;  
    int CurrentScore;  
    int RewardCounter;  
    int PowerUpCount;  
    int PrefTheme;  
    Node* friendsHead;  
};
```

F. Friends System Class

```
class FriendsPage {  
    Player* allPlayers;  
    int totalPlayers;  
    string pendingRequests[50][2];  
    int requestCount;  
    Node* hashTable[101];  
};
```